EcoPlate

Prethel, Kumud, Reid, Tobenna , Zeel

CS 4347.004: Database Systems

Professor: Dr. Jalal Omer

December 3, 2025

TABLE OF CONTENTS

I. INTRODUCTION

“EcoPlate” is a web application designed to reduce food waste from grocery stores, restaurants, and others simply by sharing food that is still edible to many different places, mainly to charities and food banks. This is achieved via real-time tracking of the products donated, and the ability for recipients to claim the available products through a centralized database. Throughout this report, our team will cover the bases in regards to the system requirements, conceptual design of the database, schema, functional dependencies and database normalization, and many more!

II. System Requirements

1. **<u>System Architecture Diagram: 3-Tier Web Architecture</u>**
    a. Frontend (Client)
        i. Actors: Donors and Recipients
        ii. Components:
            1. HTML files: “browse.html”, “donation.html”, “index.html”, “donation-form-example.html”
            2. JS files: “index.js”, “donations.js”, “donors.js”, “products.js”, “recipients.js”, “index-updated.js”
            3. CSS: “styles.css”
    b. Backend (Server)
        i. Core - Using a Node.js environment with the Express.js framework
        ii. Used to communicate with the database
    c. Database (Data) - database called “ecoplate_db” in MySQL
2. **<u>Interface Requirements:</u>**
    a. Web-based
        i. Donor Interface: contains a form for the actor to fill in their personal information such as their name and email as well as the product details
        ii. Recipient Interface - displays a list of available products (name, category, quantity, and expiration date). Grants the actor to either request a certain amount of food, delete the food from the list, and edit the quantity for a product.
    b. API

i. Endpoints:
    1. /api/products
        a. Managing inventory
    2. /api/donations
        a. Processing the donation transactions
    3. /api/donors
        a. Managing donor information
    4. /api/recipients
        a. Managing recipient information and food requests

c. Database Interface - utilizing a MySQL database
    i. Requires authentication with a host, user, password, database name, and port number.

3. <u>Functional & Non-functional Requirements of the Database System</u>

a. Functional:
    i. Donor Management
        1. The system must allow the creation of new donor profiles with a name, phone number, location, and email address
        2. The system must validate user input, especially their email and the quantity given for the product (must be greater than 0).
    ii. Recipient Management:
        1. The system must allow the recipients to send in requests for available products

2. The system must display a list of available food with their quantities attached as well to the recipients

iii. Inventory Management:

1. The system must grant the donors the ability to insert new food products with their quantity and expiration date
2. The system must update the product details upon deletion, edit, and request.

iv. Donation Management

1. The system must provide a linkage between the donations and donors.
2. The system must track the status of the donation (such as “pending”, “available”, “cancelled”, and “completed”)

b. Non-functional:

i. DATA INTEGRITY AND CONSISTENCY:

1. The database must enforce referential integrity via the foreign keys.
2. The database must follow the “ON DELETE CASCADE” constraints. Deleting a row in the Donor table must also delete any other rows in other relations associated with it.
3. The database must enforce each email address in the Donors and Recipients tables to be unique

ii. SECURITY:

1. The system must be well protected against SQL injection attacks by using prepared statements.

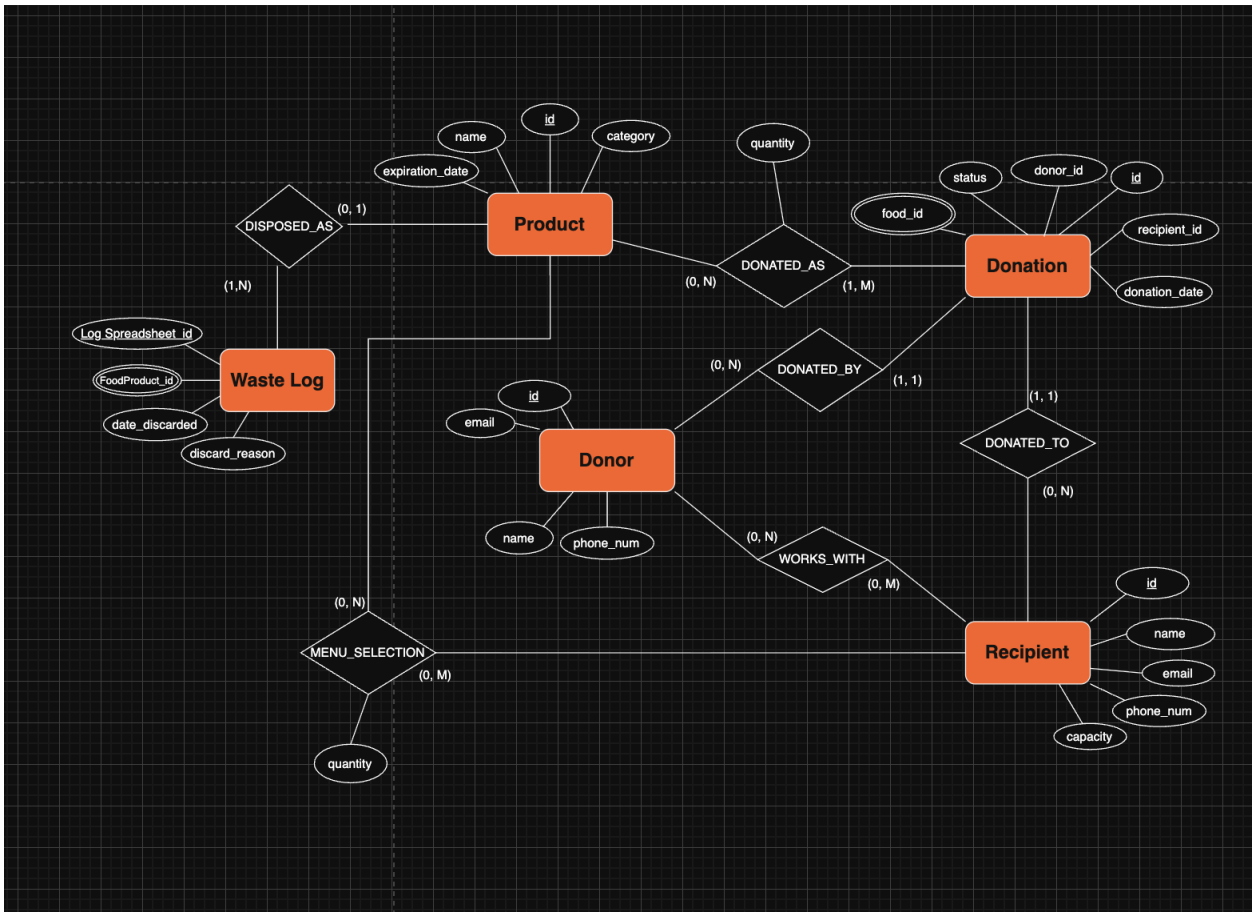
2. None of the database credentials should be hardcoded, but rather stored in the environment configuration file.

iii. PERFORMANCE:

1. The system is limited to only 10 concurrent connections at a time.
2. The webpage must include pagination to limit the number of items displayed at once, therefore reducing the rendering load.

III. Conceptual Design of the Database

a) ER Model:



b) Data Dictionary:

Donor Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each donor
name	VARCHAR(50)	-	NOT NULL	-	Donor's name or organization name
phone_num	VARCHAR(20)	-	NOT NULL	-	Optional contact phone number
email	VARCHAR(100)	UNIQUE	NOT NULL	-	Unique email address if provided

Recipient Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each recipient
name	VARCHAR(50)	-	NOT NULL	-	Name of the recipient organization or contact
phone_num	VARCHAR(20)	-	NULLABLE	-	Optional contact number
email	VARCHAR(100)	UNIQUE	NULLABLE	-	Unique email address if provided
capacity	INT	-	NOT NULL	-	Maximum quantity the recipient can accept

Product Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each product
category	VARCHAR(50)	-	NOT NULL	-	Type of food
name	VARCHAR(50)	-	NOT NULL	-	Name of the product
expiration_date	DATE	-	NOT NULL	-	Date the product expires
quantity	INT	-	NOT NULL	-	Quantity of the product available

Donation Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each donation
status	VARCHAR(20)	-	NOT NULL	-	Status of the donation
donor_id	INT	FK->Donor(id)	NOT NULL	-	ID of the donor providing the donation
recipient_id	INT	FK->Recipient(id)	NOT NULL	-	ID of the donation's recipient
donation_date	DATE	-	NOT NULL	-	Date the donation is fulfilled

Waste_log Relation:

Attribute	Type	PK/FK	Null?	Default	Description
Log_Spreadsheet_id	INT	PK	NOT NULL	-	Unique identifier for each waste log spreadsheet/similar
date_discarded	DATE	-	NOT NULL	-	Date of disposal
discard_reason	VARCHAR(256)	-	NULLABLE	-	Reason for disposing of food

Works_With Relation:

Attribute	Type	PK/FK	Null?	Default	Description
donor_id	INT	FK -> Donor(id)	NOT NULL	-	Donor collaboration/communication with the recipient.
recipient_id	INT	FK -> Recipient(id)	NOT NULL	-	Recipient collaboration/comm with the donor

MENU_SELECTION Relation:

Attribute	Type	PK/FK	Null?	Default	Description
Product_id	INT	FK -> Product(id)	NOT NULL	- -	Without yet setting up donation, ID of donor providing products
recipient_id	INT	FK -> ...Recipient(id)	NOT NULL	- -	ID of recipients.. ...choosing ...the products
quantity	INT	-	NULLABLE		Associative relation, each product has quantity in this context

Product_ID Relation

Attribute	Type	PK/FK	Null?	Default	Description
FoodProduct_id	INT	FK -> Log_Spreadsheet_id FK -> Product(ID)	NOT NULL	-	Multi-valued attribute to show wastelog can hold multiple products, each having a quantity in terms of what was thrown out

Food_id Relation

Attribute	Type	PK/FK	Null?	Default	Description
Food_id	INT	Food_id -> Donation(id) FK -> Product(ID)	NOT NULL	-	Multi-valued attribute to show each donation can hold multiple products, Is referring to products

Donated_as Relation

Attribute	Type	PK/FK	Null?	Default	Description
Product_id	INT	Product_id -> Product(id)	NOT NULL	-	Connects Product to Donation
Donation_id	INT	FK -> Donation(id)	NOT NULL	-	Connects Donation to product
quantity	INT	-	NULLABLE	-	Shows each product has a quantity in the context of donations and because it is associative relation

Foreign Key Delete Rules:

- donor_id → Donor(id): *ON DELETE CASCADE, for every foreign key*

c) Business Rules:

i) Cardinality

1) Donor ↔ Donation
    (a) A Donor can make 0 or N Donations.
    (b) A Donation must be created by 1 Donor.
2) Recipient ↔ Donation
    (a) A Recipient can receive 0 or N Donations
    (b) A Donation must be sent to 1 Recipient
3) Product ↔ Donation
    (a) A Product can be part of 0 or N Donations.
    (b) A Donation must have at least 1 Product.
4) Donor ↔ Recipient
    (a) A Donor can work with 0 or N Recipients.
    (b) A Recipient can work with 0 or N Recipients.
5) Product ↔ Waste Log
    (a) A Product can be either 0 or 1 Waste Log.
    (b) A Waste Log must include at least 1 Product.

ii) Data Integrity Constraints

1) Referential Integrity:
    (a) If a Donor is deleted from the system, all of their donations are also deleted automatically
    (b) If a recipient is deleted from the system, all of the donations and other records are deleted automatically
    (c) A Product being deleted results in the records in “Donated_As” being deleted as well.

IV. Logical Database Schema

1. Schema

```
-- Donor table
CREATE TABLE Donor(
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   phone_num VARCHAR(20) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL
);

-- Recipient table
CREATE TABLE Recipient(
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   phone_num VARCHAR(20),
   email VARCHAR(100) UNIQUE,
   capacity INT NOT NULL
);

-- Product table
CREATE TABLE Product(
   id INT AUTO_INCREMENT PRIMARY KEY,
   category VARCHAR(50) NOT NULL,
   name VARCHAR(50) NOT NULL,
   expiration_date DATE NOT NULL,
   quantity INT NOT NULL
```

```
);

-- Donation table
CREATE TABLE Donation(
   id INT AUTO_INCREMENT PRIMARY KEY,
   status VARCHAR(20) NOT NULL,
   donor_id INT NOT NULL,
   recipient_id INT NOT NULL,
   donation_date DATE NOT NULL,
   FOREIGN KEY (donor_id) REFERENCES Donor(id) ON
DELETE CASCADE,
   FOREIGN KEY (recipient_id) REFERENCES Recipient(id)
ON DELETE CASCADE
);

-- Waste_log table
CREATE TABLE Waste_log(
   Log_Spreadsheet_id INT AUTO_INCREMENT PRIMARY
KEY,
   date_discarded DATE NOT NULL,
   discard_reason VARCHAR(256)
);

-- Works_With table
CREATE TABLE Works_With(
   donor_id INT NOT NULL,
   recipient_id INT NOT NULL,
   PRIMARY KEY (donor_id, recipient_id),
```

```
  FOREIGN KEY (donor_id) REFERENCES Donor(id) ON
DELETE CASCADE,
  FOREIGN KEY (recipient_id) REFERENCES Recipient(id)
ON DELETE CASCADE
);

-- MENU_SELECTION table
CREATE TABLE MENU_SELECTION(
  Product_id INT NOT NULL,
  recipient_id INT NOT NULL,
  quantity INT,
  PRIMARY KEY (Product_id, recipient_id),
  FOREIGN KEY (Product_id) REFERENCES Product(id) ON
DELETE CASCADE,
  FOREIGN KEY (recipient_id) REFERENCES Recipient(id)
ON DELETE CASCADE
);

-- Product_ID table
CREATE TABLE Product_ID(
  FoodProduct_id INT NOT NULL,
  Log_Spreadsheet_id INT NOT NULL,
  PRIMARY KEY (FoodProduct_id, Log_Spreadsheet_id),
  FOREIGN KEY (FoodProduct_id) REFERENCES
Product(id) ON DELETE CASCADE,
  FOREIGN KEY (Log_Spreadsheet_id) REFERENCES
Waste_log(Log_Spreadsheet_id) ON DELETE CASCADE
);
```

```
-- Food_id table
CREATE TABLE Food_id(
   Food_id INT NOT NULL,
   Donation_id INT NOT NULL,
   PRIMARY KEY (Food_id, Donation_id),
   FOREIGN KEY (Food_id) REFERENCES Product(id) ON
DELETE CASCADE,
   FOREIGN KEY (Donation_id) REFERENCES Donation(id)
ON DELETE CASCADE
);

-- Donated_as table
CREATE TABLE Donated_as(
   Product_id INT NOT NULL,
   Donation_id INT NOT NULL,
   quantity INT,
   PRIMARY KEY (Product_id, Donation_id),
   FOREIGN KEY (Product_id) REFERENCES Product(id) ON
DELETE CASCADE,
   FOREIGN KEY (Donation_id) REFERENCES Donation(id)
ON DELETE CASCADE
);
```

2. **Expected Database Operations and Estimated Data Values:**
    a. **Expected Database Operations:**
        i. **Insertions (high rate)**
            1. **Addition to inventory**
            2. **New donation records**

3. **New records in waste-log**

ii. **Queries (high rate)**

1. **Inventory check**
2. **Donation History**

iii. **Updates (normal rate)**

1. **Updating quantity of a product**
2. **Updating the status of a product**

b. Estimated Data Values

i.

Table	Initial Rows	Monthly Growth
Donor	20	3
Recipient	10	1
Product	150	500
Donation	50	100
Donation_As	200	350
Waste_log	15	20

V. Functional Dependencies and Database Normalization

Donor Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each donor
name	VARCHAR(50)	-	NOT NULL	-	Donor's name or organization name
phone_num	VARCHAR(20)	-	NOT NULL	-	Optional contact phone number
email	VARCHAR(100)	UNIQUE	NOT NULL	-	Unique email address if provided
location	VARCHAR(100)	UNIQUE	NOT NULL	-	A restaurant or grocery's location

1. **{id} -> {name, phone_num, email, location}**
2. **{email} -> {id, name, phone_num, location}**
3. **{location} -> {id, name, phone_num, email}**

Recipient Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each recipient
name	VARCHAR(50)	-	NOT NULL	-	Name of the recipient organization or contact
phone_num	VARCHAR(20)	-	NULLABLE	-	Optional contact number

email	VARCHAR(100)	UNIQUE	NULLABLE	-	Unique email address if provided
capacity	INT	-	NOT NULL	-	Maximum quantity the recipient can accept
location	VARCHAR(100)	UNIQUE	NOT NULL	-	We will assume recipients are just food shelters, thus values don't repeat in any row

1. **{id} -> {name, phone_num, email, capacity, location}**
2. **{location} -> {id, name, phone_num, email, capacity}**

Product Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each product
category	VARCHAR(50)	-	NOT NULL	-	Type of food
name	VARCHAR(50)	-	NOT NULL	-	Name of the product
donor_id	INT	FK - > Donor(id)	NOT NULL	-	Each donor can have multiple products
expiration_date	DATE		NOT NULL	-	Date the product expires
quantity	INT	-	NOT NULL	-	Quantity of the product available

1. **{id} -> {donor_id, category, name, expiration_date, quantity}**

Donation Relation:

Attribute	Type	PK/FK	Null?	Default	Description
id	INT	PK	NOT NULL	-	Unique identifier for each donation
status	VARCHAR(20)	-	NOT NULL	-	Status of the donation
donor_id	INT	FK->Donor(id)	NOT NULL	-	ID of the donor providing the donation
recipient_id	INT	FK->Recipient(id)	NOT NULL	-	ID of the donation's recipient
donation_date	DATE	-	NOT NULL	-	Date the donation is fulfilled

1. **{id} -> {status, donor_id, recipient_id, donation_date}**

Waste_log Relation:

Attribute	Type	PK/FK	Null?	Default	Description
Log_Spreadsheet_id	INT	PK	NOT NULL	-	Unique identifier for each waste log spreadsheet/similar

date_discarded	DATE	-	NOT NULL	-	Date of disposal
discard_reason	VARCHAR(256)	-	NULLABLE	-	Reason for disposing of food

1. **{Log_Spreadsheet_id} -> {date_discarded, discard_reason}**

Works_With Relation:

Attribute	Type	PK/FK	Null?	Default	Description
donor_id	INT	pk,FK -> Donor(id)	NOT NULL	-	Donor collaboration/communication with the recipient.
recipient_id	INT	pk,FK -> Recipient(id)	NOT NULL	-	Recipient collaboration/comm with the donor

1. **{donor_id, recipient_id} is pk , no nontrivial dependencies**

MENU_SELECTION Relation:

Attribute	Type	PK/FK	Null?	Default	Description
Product_id	INT	FK -> Product(id)	NOT NULL	- -	Without yet setting up donation, ID of donor providing products
recipient_id	INT	FK -> ...Recipient(id)	NOT NULL	- -	ID of recipients.. ...choosing ...the products
quantity	INT	-	NULLABLE		Associative relation, each product has

					quantity in this context

1. **Primary key: {Product_id, recipient_id}**
2. **{Product_id, recipient_id} -> quantity**

WasteLog_Productbridge

Attribute	Type	PK/FK	Null?	Default	Description
Log_Spreadsheet_id	INT	Pk, FK -> Waste_log(Log_Spreadsheet_id)	NOT NULL	-	Multi-valued attribute to show wastelog can hold multiple products, each having a quantity in terms of what was thrown out
Product_id	INT	Pk, FK -> Product(id)	NOT NULL		identifies which specific product batch is being thrown away
quantity_discarded	INT	-	NULLABLE		

1. **Primary key: {Log_Spreadsheet_id, Product_id}**
2. **{Log_Spreadsheet_id, Product_id} -> quantity_discarded**

Donated_as Relation

Attribute	Type	PK/FK	Null?	Default	Description
Product_id	INT	Product_id -> Product(id)	NOT NULL	-	Connects Product to Donation
Donation_id	INT	FK -> Donation(id)	NOT NULL	-	Connects Donation to product

quantity	INT	-	NULLABLE	-	Shows each product has a quantity in the context of donations and because it is associative relation

1. **{Donation_id, Product_id} -> {quantity}**

VI. User Application Interface

1. **Tech Stack**
    a. HTML files
    b. CSS for styling
    c. Javascript for dynamic logic
2. **User Interaction**
    a. Donor Page
        i. The user clicks on the “Donate” link in the navigation bar or the green button labeled as “Start Donating”
        ii. The donor fills out a form with their contact information and the product information.
        iii. By clicking on submit, their form is then processed in the backend side of the project.
        iv. They are finally alerted on a successful donation.
    b. Recipient Page
        i. The user clicks on the “Browse” link in the navigation bar or the green button labeled as “Browse Available Donations”
        ii. The page contains multiple cards for every available product, each showing the category, product name, quantity, and expiration date.
        iii. The user can click on the “Request” button for a specific product. This will prompt the user for a valid quantity

VII. Conclusion and Future Work

1. **Conclusion** - EcoPlate ended in a success! Although our web application seems basic in terms of aesthetic, it succeeds in accomplishing our main goal, that is granting a hub between the donor and recipients.
2. **Future Work -** This project would be majorly improved if we implemented some sort of user authentication. In the recipient page, anyone could easily edit the cards, or even worse delete them even though they never sent in the donations in the first place. Requiring the users to login will decrease the risk of facing malicious attacks. Another improvement would be implementing a way to notify potential recipients of new items being posted. That way, recipients won't have to constantly search for new items every second.

VIII. References

Omer, J. (2025, December 5). *SQL Injections* [PowerPoint slides]. CS 4337: Database Systems, University of Texas at Dallas. SQL Injections.pptx

Omer, J. (2025, December 5). *Chapter 6: Basic SQL* [PowerPoint slides]. CS 4337: Database Systems, University of Texas at Dallas. ch06 Basic SQL.pptx

IX. Appendix

The title “Appendix” is linked to a downloadable ZIP file. Please click on it for easy access.